Abstract diagrams for the working mathematician: The Asymptote `smoothmanifold` module

Roman Maksimovich

Abstract

Within the niche of mathematicians, there's a niche of those who need to draw diagrams related to pure mathematics. Within *that* niche there's a niche of people who have experience in programming and feel comfortable writing code to produce graphics. This article is written with this niche in mind. It's a survey of the available tools and a showcase of a library I wrote specifically to simplify the process of drawing diagrams in differential geometry, topology, set theory, etc.

1 Introduction

Back in high school, I was attending a seminar on differential geometry, and I wanted to practice my LaTeX skills by typesetting the lecture notes. I found myself needing to draw simple surfaces such as those seen in Figure 1.

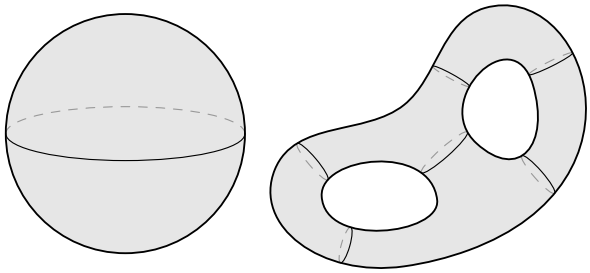


Figure 1: Basic two-dimensional manifolds

At that point I realized I wasn't sure how to proceed. I started by considering some of the available tools:

- **Inkscape** (`www.inkscape.org`)
  This is a popular and versatile GUI program for drawing vectorized graphics, suitable when the diagram is simple enough to be drawn by hand with some precision assistance. This assumption holds for the left-hand side of Figure 1, but not for the right-hand side. The reason is that each "cross section ring" has to be drawn at a precisely calculated position, where it meets its bounding paths at equal angles, as illustrated in Figure 2.

  Further, it would be useful to tweak the count and positions of these "cross sections" to see what works best for the picture, and this is not something that can be achieved with a GUI tool like Inkscape. Pass.

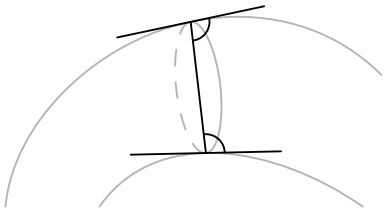


Figure 2: A good position for drawing a cross section ring, equal angles with the tangent lines

- **LaTeX's `picture` environment**
  There are native drawing capabilities in LaTeX, but they are severely limited. For instance, a straight line segment can be drawn like so:

```
\begin{picture}(15,30)
  \put(0,0){\line(1,2){15}}
\end{picture}
```

  This yields the line shown on the right-hand side. The command `\line(m,n){l}` creates a line whose slope is `n`/`m` and whose horizontal projection is `l` units long. However, both `n` and `m` must not exceed 6, and must be coprime. The reason for these weird restrictions is the mechanism behind the `picture` environment: instead of producing true vector graphics, it tries to assemble the diagram from glyphs found in TeX's fonts, and these fonts only provide a small variety of sloped straight lines (if you are reading this article digitally, with some PDF readers you can zoom in on the straight line above and see that it's actually three aligned forward slashes).

  To be fair, `picture` is also capable of drawing an arbitrary Bézier curve by putting it together from many black squares. Still, this is a pretty rudimentary tool unsuited for complicated technical drawing, and it's mainly used to define custom symbols rather than full diagrams. Pass.
- **The Ti*k*Z package** (`tikz.dev`)
  For a majority of TeX users, this is probably the go-to tool for technical drawing, and reasonably so. Ti*k*Z has a comprehensive manual and a rich ecosystem of libraries to suit virtually your every need. Clearly, both manifolds on Figure 1 can be drawn with Ti*k*Z.

  However, my high school self was positively intimidated by the Ti*k*Z manual (1321 pages?!), and besides, I wanted to write an *algorithm* that could calculate the best positions for cross section rings, not just a TeX macro. I was fascinated by the C programming language at the time, and I wanted to write a *function* which would accept two paths as arguments, as well as

the desired number of cross sections, and return the coordinates at which they would look best. TEX certainly tries to provide some features of a full programming language, but in truth it is more of a markup language, poorly suited for writing general-purpose algorithms.

Hence, I passed on Ti*k*Z also. Alternatives to Ti*k*Z (such as `pstricks`, `xypic`, `dratex`, etc.) suffer from the same and more disadvantages.

- METAPOST (`tug.org/metapost.html`)
  Introduced by John D. Hobby in 1994, this language is based on Donald Knuth's METAFONT program and allows users to produce PostScript graphics with a set of drawing commands. Unlike Ti*k*Z and its cousins, METAPOST is a standalone complete programming language with data structures and flow control (such as `if` statements and `for` loops). However, while still perfectly usable, I would consider METAPOST to be outdated and lacking in certain features that are present in other drawing tools, such as native PDF output. And besides, METAPOST turns out to have a successor which inherits its advantages and adds many more features.

That successor is a vector graphics language called Asymptote (`asymptote.sourceforge.io`). It was first introduced in 2004 at the University of Alberta by Andy Hammerlindl, John Bowman, and Tom Prince.

Asymptote follows METAPOST in being a proper programming language with data structures, IEEE floating-point number support, and control flow. But unlike METAPOST, it has a C-like syntax and some very high-level features such as default arguments and first-class functions (i.e., functions that can be returned or passed to other functions as values). Readers of *TUGboat* may have already read about Asymptote in [1] and [3].

Soon after learning about Asymptote, I started writing my own library for this language, one that I later came to use whenever I needed to create a diagram in abstract mathematics.

2 The `smoothmanifold` module

2.1 The basics of Asymptote

An Asymptote user produces graphics by writing code in a file with the `.asy` extension, then invoking the `asy` program on the command line to compile the code to a picture in a format such as PDF, SVG, PostScript, or a number of others. Additionally, one may write Asymptote code directly in a LaTeX document by means of the `asymptote` package. This approach removes the necessity to manually include images, but the code becomes harder to isolate.

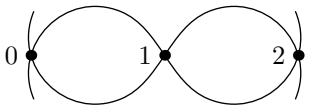


Figure 3: A sample picture produced by Asymptote

An example code snippet might look like this:

```
settings.outformat = "eps";
size(4cm);
path p = (0,1) .. (1,0) .. (2,1) .. (3,0);
path q = (0,0) .. (1,1) .. (2,0) .. (3,1);
draw(p); draw(q);
pair[] points = intersectionpoints(p, q);
for (int i = 0; i < points.length; ++i) {
  dot(Label((string)i, align = W),
      points[i], linewidth(4pt));
}
```

The result of compiling this code is an Encapsulated PostScript file, shown in Figure 3. Note Asymptote's support of arrays and its native function `intersectionpoints` (whereas a Ti*k*Z user would load additional libraries to gain this functionality).

Asymptote has other outstanding features such as powerful three-dimensional drawing, rich support for mathematical functions, but most importantly, the ease with which the language can be extended or modified. In the following subsections, I will outline the most prominent features of my own library (which now comes to more than 6000 lines of code) and how they allow for effortless drawing of abstract diagrams.

2.2 Smooth objects and cross sections

As you recall, my original goal was drawing two-dimensional manifolds like those in Figure 1, so this was the first functionality I programmed into my library, named `smoothmanifold`. A simple torus can be drawn by writing

```
import smoothmanifold;
smooth s = smooth(
  ellipse((0,0), 2, 1)
).addhole(
  ellipse((0,0), 1.1, .37),
  sections = new real[][]{
    r(0, 60, 3), r(90, 90, 1),
    r(180, 60, 3), r(270, 90, 1)
  }
);
config.section.freedom = .9;
size(5cm);
draw(s, dpar(
  mode = free, viewdir = 3.0*dir(90)
));
```

Compiling this code (assuming that the file `smoothmanifold.asy` is located where Asymptote can find it) yields the picture in Figure 4.

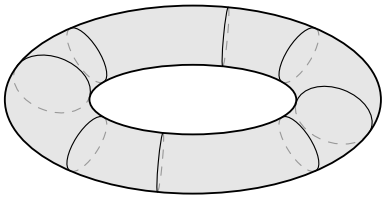


Figure 4: A torus

You can see that the code is relatively high-level: instead of drawing cross section rings manually, we build a *smooth object* with the `smooth` constructor, then add a hole to it with `addhole`, whose `sections` parameter allows configuring the positions of cross sections. It's not crucial for the reader to understand what exactly goes on in that `real[][]` array; suffice it to say that it tells `smoothmanifold` roughly where we want it to draw cross section rings around the given hole.

After creating the smooth object and assigning it to the variable `s`, we set some configuration values. The `config.section.freedom` value determines how much leeway is given to the section search algorithm (to be explained later), and the `size()` function allows us to set the picture's size.

Finally, we `draw` the smooth object, providing a `dpar` (**d**rawing **par**ameters) structure, where we specify how the object (and its cross sections) should be drawn. The `viewdir` parameter is the "direction of view", and it helps coordinate the widths of different cross section rings, such that together they create the illusion of a 3D object.

Interested readers may consult page 22 of the full `smoothmanifold` manual [4] for more detail.

2.3 The cross section search algorithm

Although I don't wish to delve too deeply into technical aspects, I think the cross section search algorithm merits a short discussion. The relevant `smoothmanifold` function is

```
pair[][] sectionparams
(path g, path h, int n, real r, int p)
```

It tries to find the best positions for `n` cross sections between the paths `g` and `h`, and it does so by first splitting both paths into regions, as seen in Figure 5.

Here `n` is set to 5, so five regions are selected in the space between the paths, and the algorithm will try to find an optimal position for a cross section in each region. The `real r` parameter of the `sectionparams` function controls the relative width

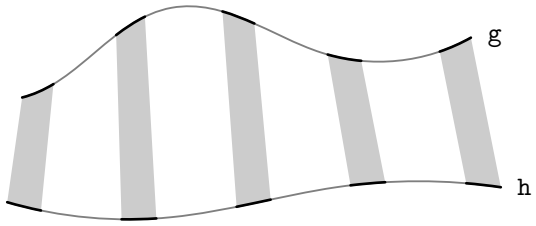


Figure 5: `n` regions (filled gray) are formed between the paths `g` and `h`

of these regions. If `r = 0`, each region is a line, and there is no choice. If `r = 1`, then the regions cover all space between `g` and `h`. In a sense, `r` is the "freedom of choice", hence its default value is the `config.section.freedom` settings option.

Then, the algorithm proceeds to select `p` sample points along either edge of each region, and search for the optimal section position by minimizing a loss function. This is illustrated in Figure 6, with `p` equal to 10.

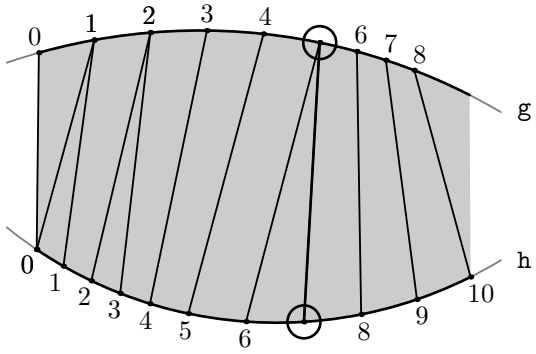


Figure 6: Selecting the optimal section position (circled) within one region

As you can see, not every combination of points is checked. This is a heuristic optimization based on the fact that "good" cross sections usually connect points whose indices are close. This allows the algorithm to run in $O(np)$ time, whereas the naïve implementation is $O(np^2)$.

One final note on the cross section mechanism concerns this "illusion of 3D". Asymptote has full support for actual 3-dimensional drawing and allows producing something like the torus in Figure 7 with relatively little effort. In fact, Asymptote even allows to embed vectorized 3D objects, either through Adobe's PRC format or (since very recently) via a custom V3D format [2]. It certainly looks more realistic, but I see two objections to this approach:

- 3-dimensional drawing in Asymptote typically requires a bit more work, since one has to define surfaces either with a mathematical function or as a surface of revolution, and one has to

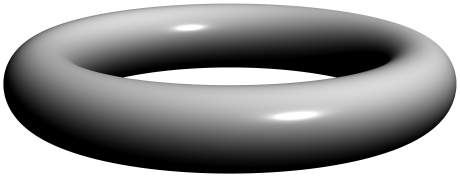


Figure 7: A real 3-dimensional torus

consider lighting and perspective. In Figure 4, a torus is just two ellipses.

- In my personal opinion, the 3-dimensional torus in Figure 7 lacks some sort of "textbook style" that is kept in Figure 4.

2.4 Smooth objects as sets

Having achieved my initial goal of drawing smooth manifolds, I forgot about the library for about a year. But one day I realized that the `smooth` structure can be used to model not only smooth manifolds, but *sets* in general. Consider the set-theoretic diagram in Figure 8 that illustrates the intersection of two sets.

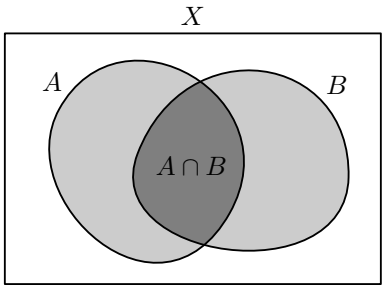


Figure 8: Intersection of two subsets of a set X

Of course, this diagram can be drawn with ease in plain Asymptote or any other graphics program, but there is a common pattern that can be abstracted away: one encompassing cyclic path (labeled X in Figure 8) and a number of cyclic paths inside of it, representing subsets and their unions/intersections. Using the current version of `smoothmanifold`, the diagram in Figure 8 is generated with this code:

```
smooth sm = smooth(
    (-1.5,-1) -- (1.5,-1)
    -- (1.5,1) -- (-1.5,1) -- cycle,
    label = "$X$", labeldir = dir(90)
).addsubset(
  convexpaths[2],
  scale = .8, shift = (-.4,.05),
  label = "$A$", dir = dir(140)
).addsubset(
  convexpaths[3],
  scale = .8, shift = (.4,0),
  label = "$B$", dir = dir(40)
).setlabel(2, "$A \cap B$", dir = (0,0));
draw(sm, dpar(
  fill = false,
  drawnow = true,
  subsetfill = new pen[]{gray(.8),gray(.5)}
));
```

The `smooth` structure has a field called `subsets` which holds an array of "subsets" and which can be added to by invoking the `addsubset` method. Most importantly, after adding the second subset, the intersection of the two subsets is *automatically calculated and added as a third subset*, so that we can then set its label by calling `.setlabel(2, "$A \cap B$")` (here, 2 is the index of the third subset, since arrays in Asymptote are zero-indexed).

Why is this useful? Because now "sets" and "subsets" in your diagram are no longer just paths — they are Asymptote objects, and the `smoothmanifold` module provides many useful methods and functions to manipulate these objects. For example, the `smooth` structure has a method called `move` that allows one to shift, scale and rotate a smooth object, respecting this object's `holes` and `subsets`, as well as other data such as cross sections and labels.

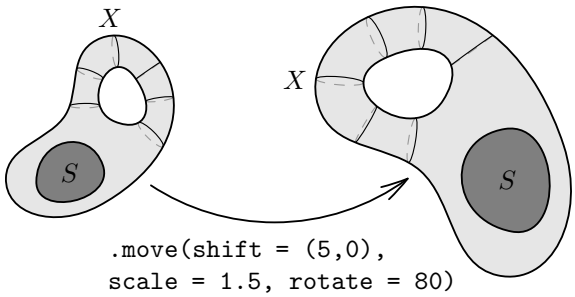


Figure 9: Calling one method to move a smooth object, along with its hole, subset, label, and cross sections

Returning to Figure 8, the automatic intersection of subsets is achieved via the `smoothmanifold` function

```
path[] intersection (
  path p, path q,
  bool round, real roundcoeff
)
```

It takes two *cyclic* paths `p` and `q` as arguments and returns the paths that bound the intersection of the areas bounded by `p` and `q`. Additionally, the intersection can be "rounded at the corners" if the `round` switch is set to `true`, and the degree of rounding can be regulated with `roundcoeff`. An example can be seen in Figure 10.

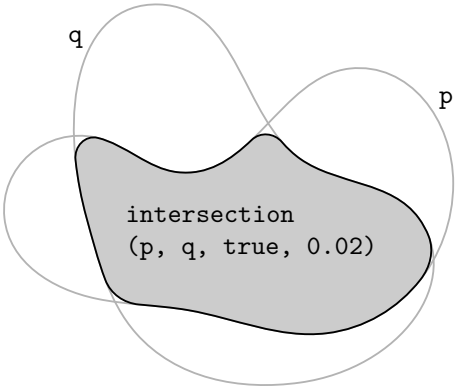


Figure 10: Intersection of two paths with rounded corners

Similarly, `smoothmanifold` contains the functions `union`, `difference`, and `symmetric` to calculate the union, difference, and symmetric difference of two areas. This can be useful not only to automatically construct subset intersections, but in general to produce new paths from old.

Side note: Asymptote already has a module called `roundedpath` that allows rounding the corners of a path; `smoothmanifold` does not depend on it, instead implementing rounding directly.

2.5 Delayed drawing of paths

One of Asymptote's celebrated features is so-called "deferred drawing". Essentially, whenever a `path` is drawn by Asymptote, it is stored in a structure of type `picture`. In particular, the `draw` function has an optional argument `picture pic`, whose default value is `currentpicture`, that allows the user to specify the picture on which to draw the path.

However, crucially, instead of directly storing the control points of the Bézier spline that a `path` represents, the `picture` structure stores a *function* which, when called, will draw the path. This function has additional arguments whose values may not be known at the time that the user calls `draw`. This is mainly because Asymptote allows drawing paths in "user coordinates" which then have to be transformed into true PostScript coordinates. Since this transformation depends on the size of the picture, it is not known at the time of drawing any particular path. As a result, drawing is "deferred" until a path's true coordinates are known, and the function stored inside a `picture` can be called.

However, this means that once the instruction to draw a path is stored inside a `picture`, this path becomes fully immutable. There is no way to "change a path after it's drawn", since the only way to even inspect this path is by calling the associated function, which will immediately draw the path, defeating the purpose.

At the same time, mutating a drawn path can be useful. For example, consider Figure 11, where two circles overlap, and the portion of the left circle going "under" the right circle is drawn with a dashed `pen`. Using `smoothmanifold`, this can be achieved

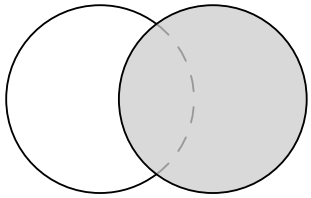


Figure 11: One circle overlapping another

with the code

```
config.drawing.underdashes = true;
config.drawing.gaplength = 0.001;
fitpath(unitcircle);
fillfitpath(
  shift(1.2) * unitcircle,
  fillpen = gray(.85)
);
```

The `fitpath` function allows the user to "draw a path on a picture and then mutate it afterwards", for example, split it into two subpaths and draw each one with different pens, as in Figure 11. It also allows leaving "gaps" in paths after they are drawn, as seen in Figure 12, which was generated using this code:

```
config.drawing.gaplength = 0.4;
fitpath(box((-3,-.6), (3,.6)));
fitpath(circle((0,0), 1.2));
```

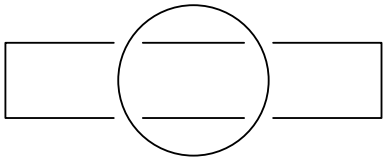


Figure 12: The circular path leaving gaps in the rectangular path

Of course, both of these pictures can be easily drawn in plain Asymptote by first assigning the paths to variables, calculating their intersection points, taking subpaths, and finally drawing the subpaths. However, this again constitutes a common pattern that can be abstracted away. And besides, what if the paths are not so easily isolated, for instance when they are the contours of smooth objects? An example is shown in Figure 13.

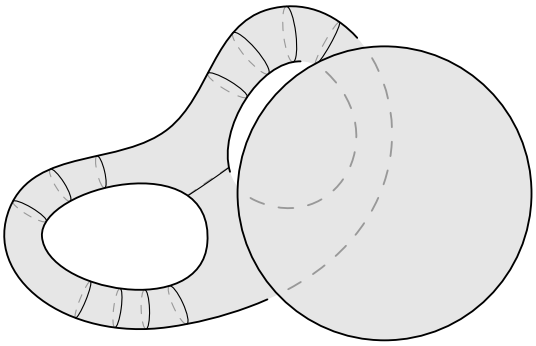


Figure 13: A circle (right) overlapping a smooth object (left) and leaving gaps in its contour, difficult to draw manually

How does the `fitpath` function work internally? In slightly simplified terms, the `smoothmanifold` module has a global variable called `delayedPaths` which holds an array of paths. When the `fitpath` function is called on a path `p`, it will traverse this array and modify the paths therein, based on their intersections with `p`. In particular, it will split existing paths into subpaths such that gaps are left wherever `p` intersects another path contained in `delayedPaths`. Finally, the path `p` itself is added to this array.

And this is essentially all that the `fitpath` function does — it doesn't actually draw anything on any `picture`. Of course, all the paths in `delayedPaths` will have to be drawn at some point. This is handled by the `drawdelayed` function which is called automatically before producing an image file from a `picture`, but can also be called manually by the user.

As you can see, this system is essentially a second layer of deferred drawing, and I call it *delayed drawing* (totally not confusing!). The true implementation is a bit more involved than my explanation here, and interested readers may see page 4 of the `smoothmanifold` manual [4] for further detail.

As a closing remark on this section, I'd like to show a real-world use case for the delayed drawing system, pictured in Figure 14. It relates to the proof of the Inverse Function Theorem from real analysis. Given a point y, we construct a certain contraction mapping on a complete metric space, and prove that its fixed point x is the unique preimage of y under F, which then allows us to construct the inverse function to F. This diagram was part of a job I did for my university, supplying illustrations to the lecture notes for a course in real analysis.

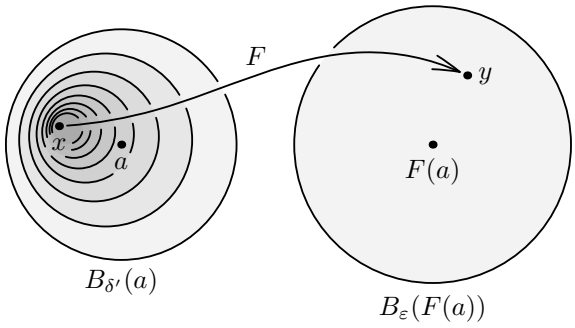


Figure 14: A construction step in the proof of the Inverse Function Theorem

3 Conclusion

For close to four years, I have been developing and maintaining the `smoothmanifold` module, and I find it very useful whenever I have to prepare a technical diagram related to higher mathematics. I cannot help but think that there are people who would benefit from this library as well.

`smoothmanifold`, like Asymptote, is free and open source software, available for download at `github.com/thornoar/smoothmanifold`.

I would also like to extend my thanks to John C. Bowman, Andy Hammerlindl, Tom Prince, and other people who developed and maintained Asymptote, for creating this graphics language and for gracefully neglecting to make many of Asymptote's core structures and functions `private`, which allows the user to perform some really dirty tricks and produce beautiful results.

References

[1] J.C. Bowman. Asymptote: Interactive TEX-aware 3D vector graphics. *TUGboat* 31(2):288–294, 2010. `tug.org/TUGboat/tb31-2/tb98bowman.pdf`, `www.math.ualberta.ca/~bowman/publications/tb98bowman.pdf`

[2] J.C. Bowman. V3D: Portable interactive 3D graphics for PDF and beyond. *TUGboat* 47(2):230–233, 2026. `tug.org/TUGboat/tb47-2/tb146bowman-v3d.pdf`, `tug.org/TUGboat/tb47-2/tb146bowman-v3d.pdf`

[3] J.C. Bowman, A. Hammerlindl. Asymptote, A vector graphics language. *TUGboat* 29(2):203–205, 2008. `tug.org/TUGboat/tb29-2/tb92bowman.pdf`, `www.math.ualberta.ca/~bowman/publications/asyTUG.pdf`

[4] R. Maksimovich. `smoothmanifold.asy`: Diagrams in higher mathematics with Asymptote. `github.com/thornoar/smoothmanifold/blob/master/documentation/main.pdf`

◇ Roman Maksimovich
Hong Kong
`r.a.maksimovich (at) gmail dot com`